The Last Clue

Computer Graphics Semester Project - Checkpoint 1: Technical Foundation

Pure C++ implementation - every pixel drawn by our own algorithms

Group members

Caleb Kipchirchir - Graphics & Algorithms (lines, circles, ellipses, transforms)

Alvin Kimani - Environment & Visual Design (polygons, filling, crime scene, characters)

Sudheysi Ali - Puzzles & Game Mechanics (clipping, point-in-polygon, safe, torn letter)

Latiffa - Interface, Animation & Integration (Bezier, splines, evidence board, Case File)

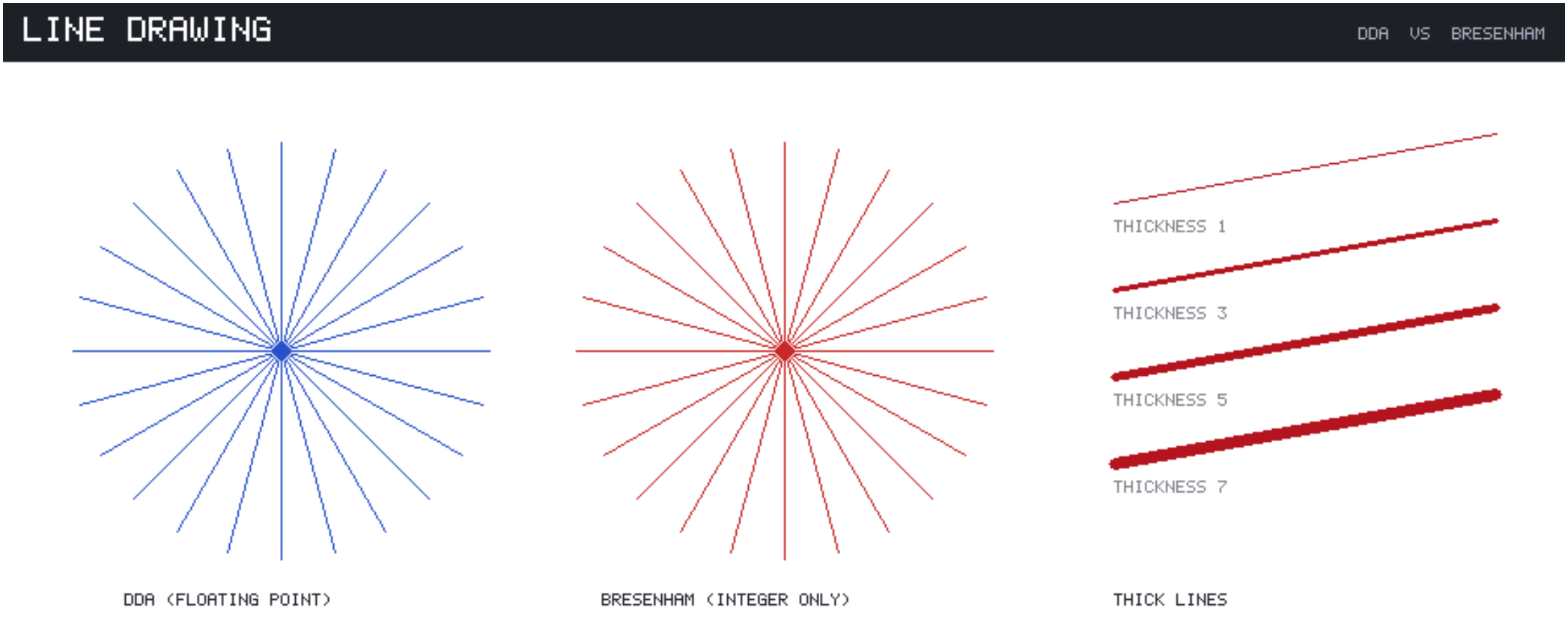
Repository

https://github.com/C-Sawe/the-last-clue



1. Line drawing: DDA vs Bresenham

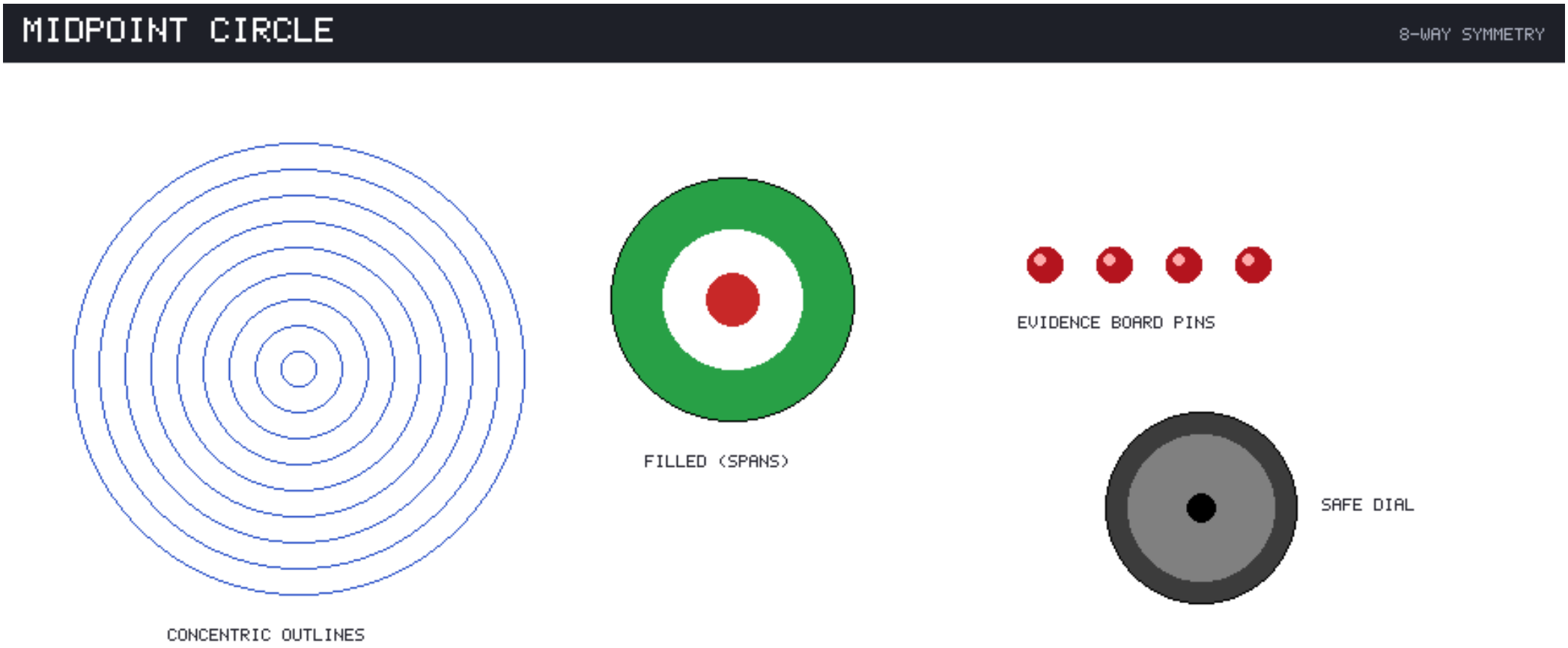
Implemented by Caleb | demos/01_lines.cpp



DDA steps with floating-point increments; Bresenham uses integers only. Both cover all 8 octants. Thick lines become evidence-board strings and wires.

2. Midpoint circle

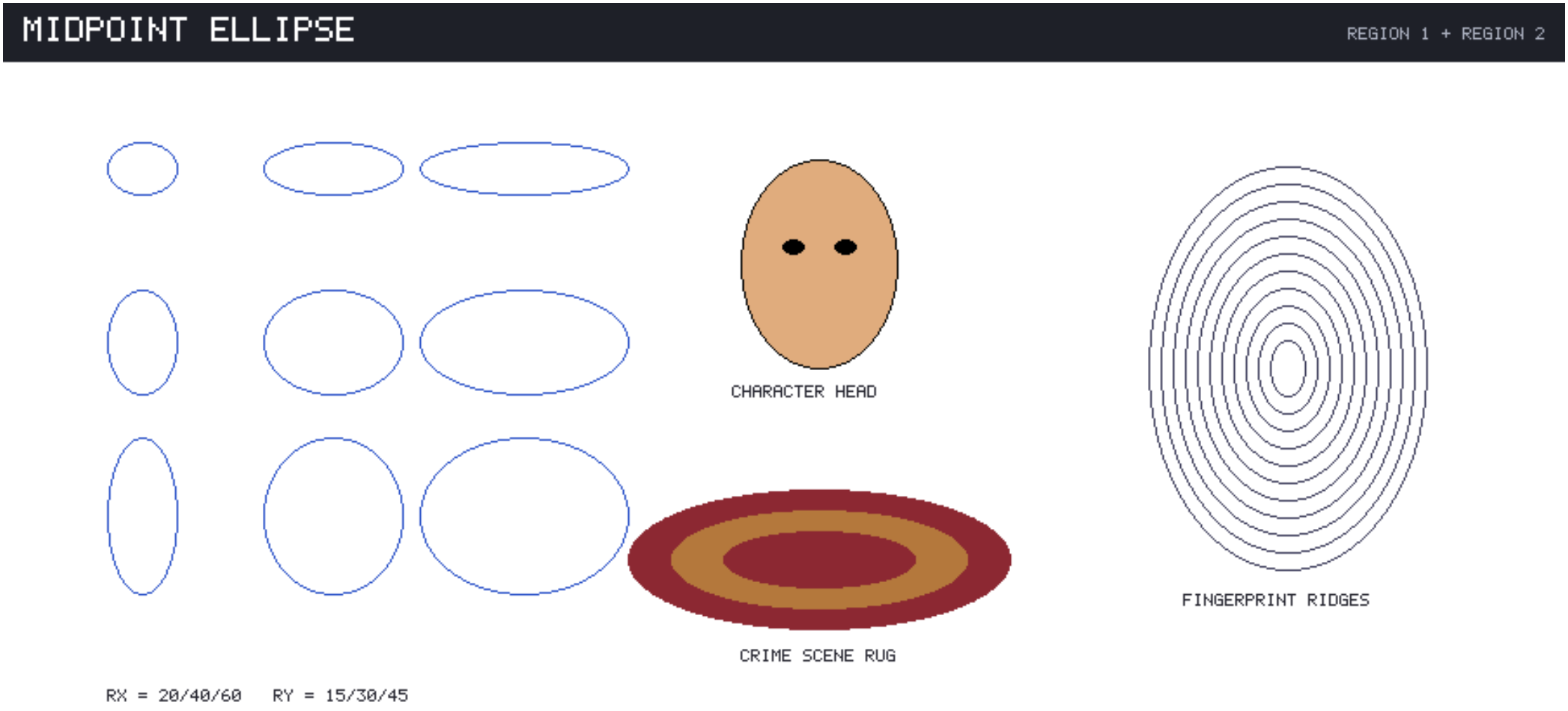
Implemented by Caleb | demos/02_circles.cpp



Decision parameter p = 1 - r with 8-way symmetry; filled circles use horizontal spans. Used for the safe dial, board pins and buttons.

3. Midpoint ellipse

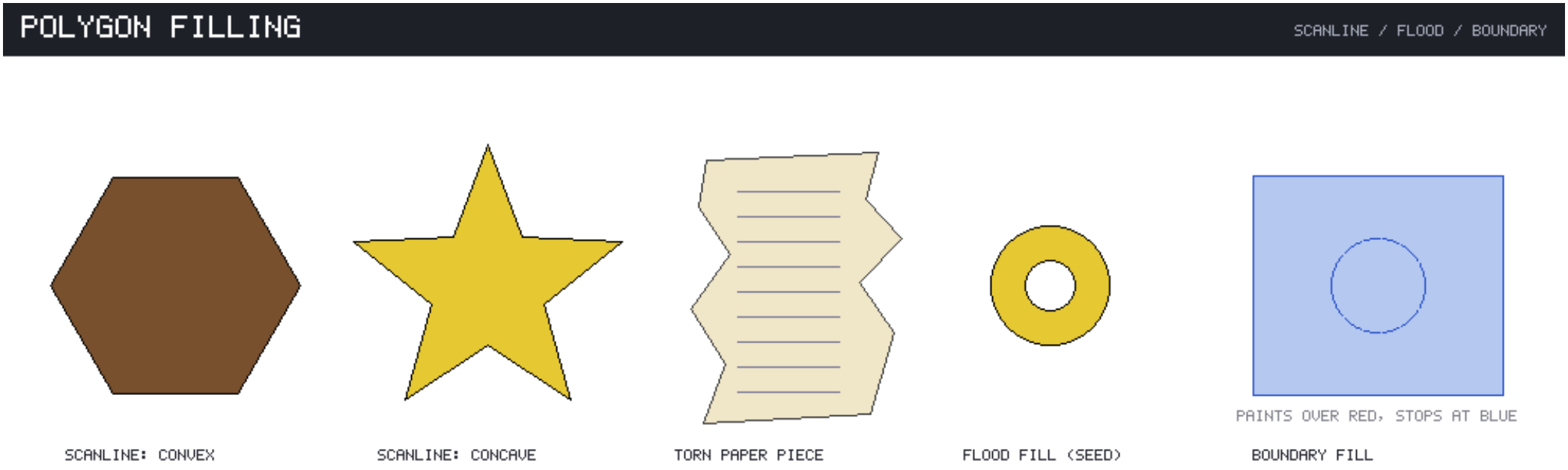
Implemented by Caleb | demos/03_ellipses.cpp



Region 1 (|slope| < 1) and region 2 (|slope| >= 1). Used for character heads, the crime-scene rug and fingerprint ridges.

4. Polygon filling

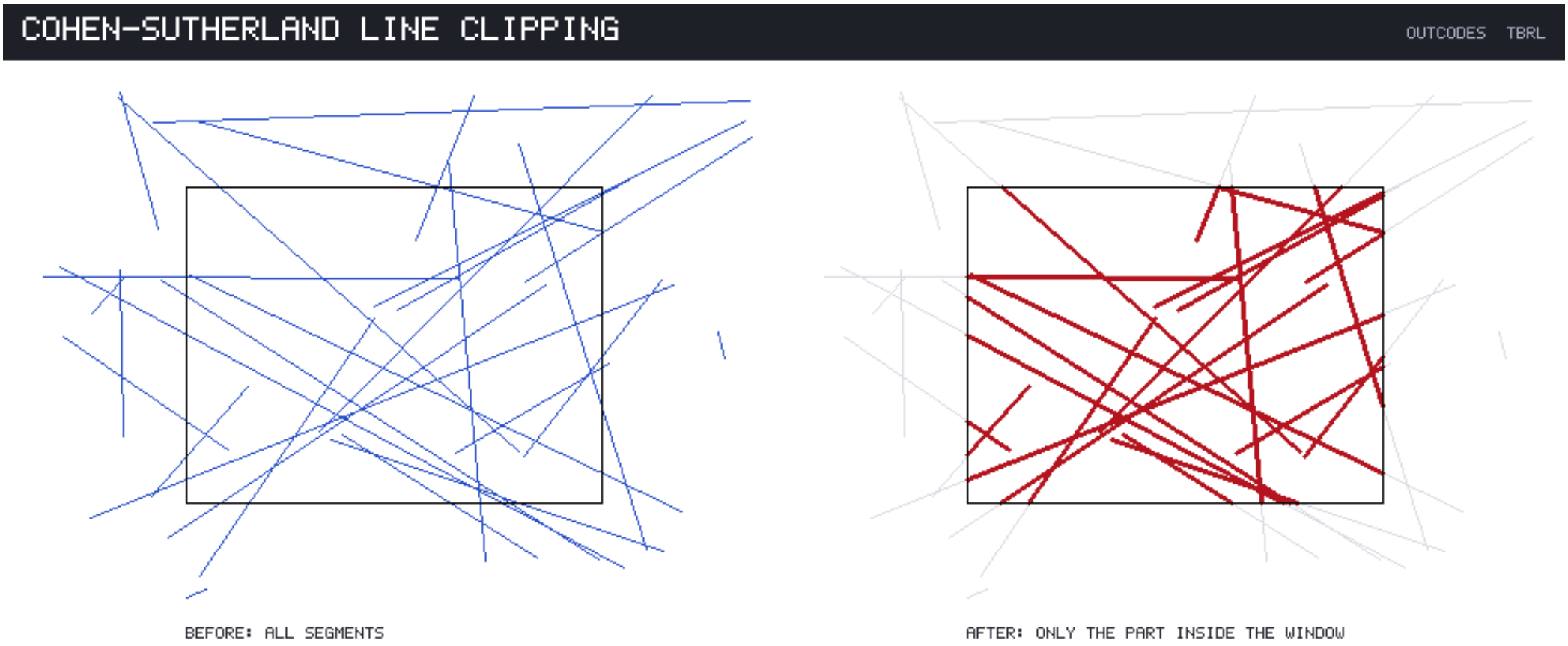
Implemented by Alvin | demos/04_fill.cpp



Scanline fill with an edge table and active edge table (convex, concave and torn-paper shapes); 4-connected flood fill and boundary fill with an explicit stack.

5. Cohen-Sutherland line clipping

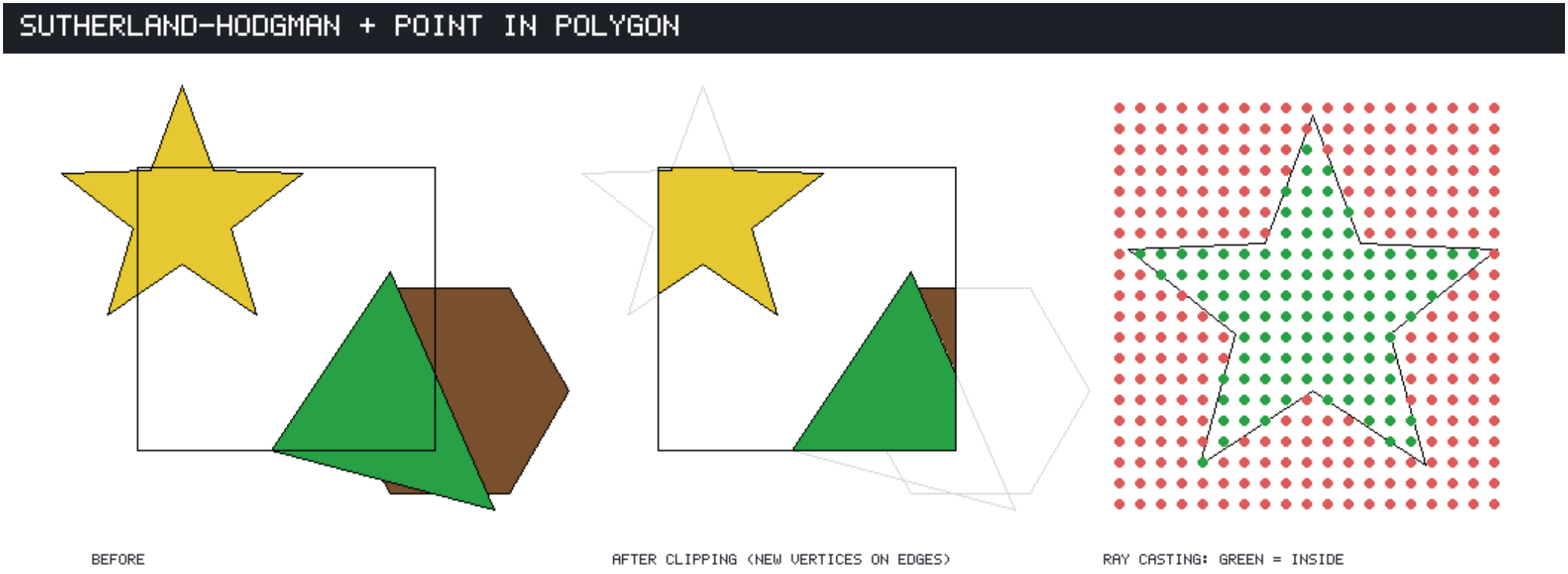
Implemented by Sudheysi | demos/05_line_clip.cpp



4-bit outcodes give trivial accept/reject; otherwise the line is intersected with the violated window edge and re-tested.

6. Sutherland-Hodgman clipping + point in polygon

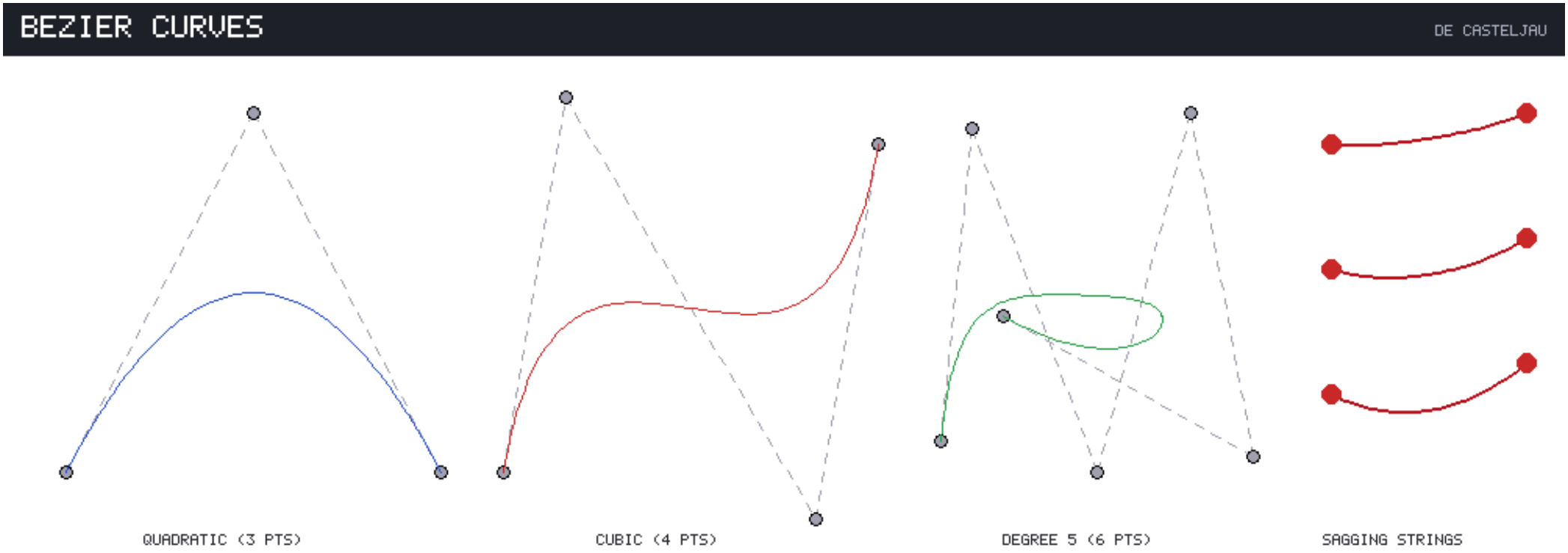
Implemented by Sudheysi | demos/06_poly_clip.cpp



Polygons are clipped against each window edge in turn. Ray casting (even-odd rule) classifies points against a concave star.

7. Bezier curves (de Casteljau)

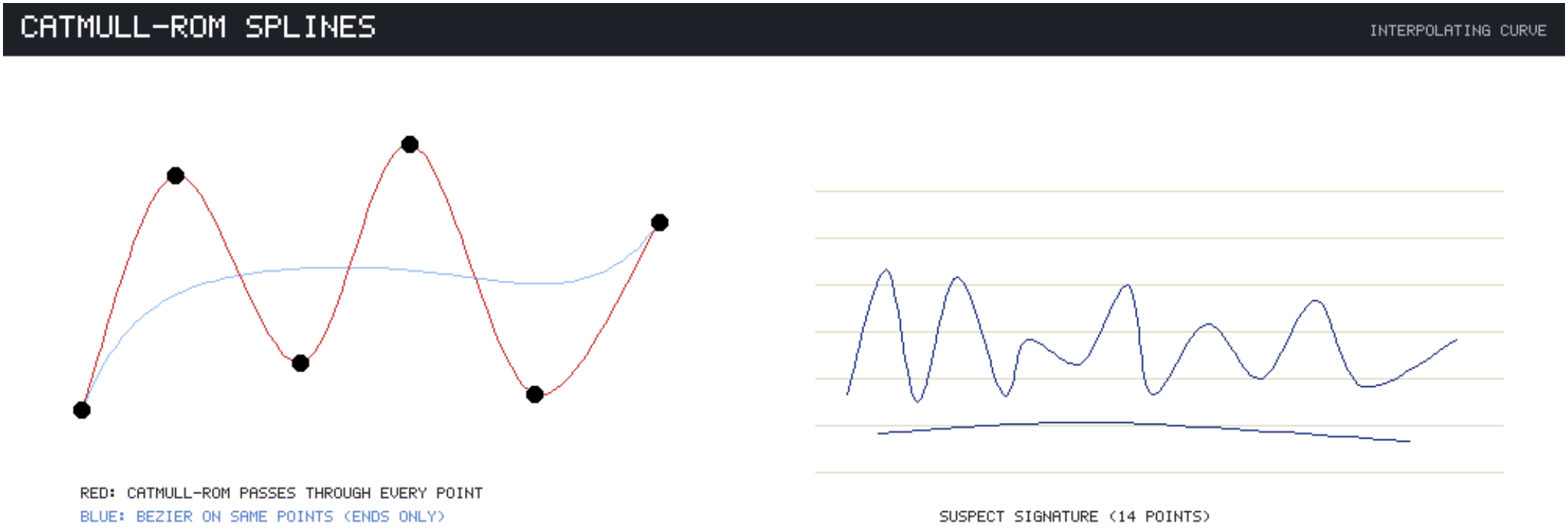
Implemented by Latiffa | demos/07_bezier.cpp



Repeated linear interpolation for any degree. A quadratic curve with a sagging control point gives the evidence-board strings.

8. Catmull-Rom splines

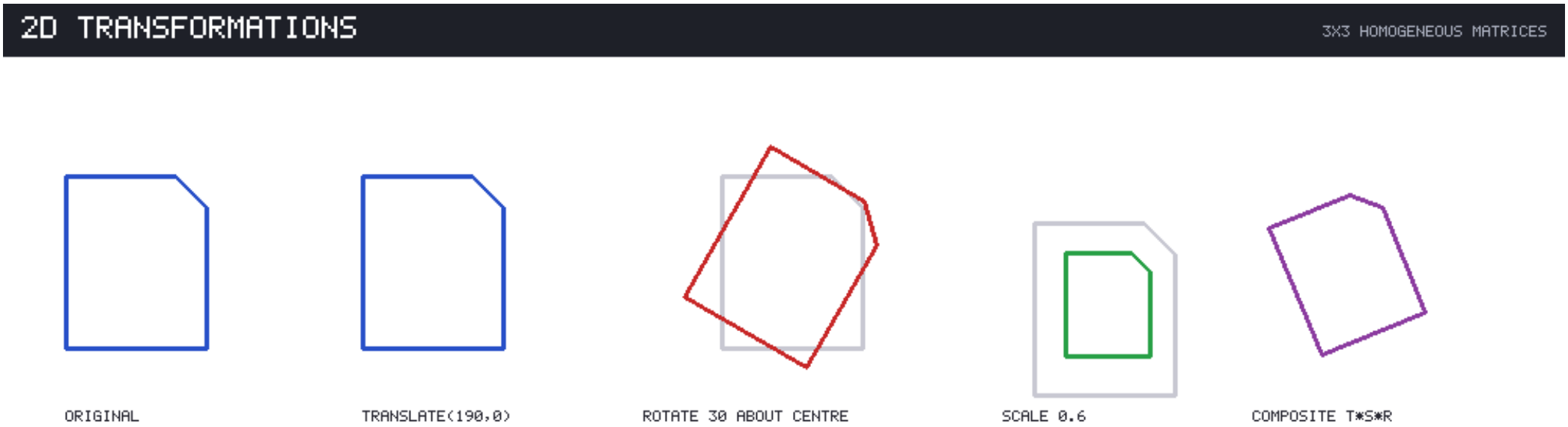
Implemented by Latiffa | demos/08_spline.cpp



Unlike Bezier, the spline passes through every point. Used for handwriting and a suspect's signature.

9. 2D transformations

Implemented by Caleb | demos/09_transforms.cpp



3x3 homogeneous matrices: translation, rotation and scaling about a pivot, and composite T*S*R as one matrix.

10. Project component: crime scene

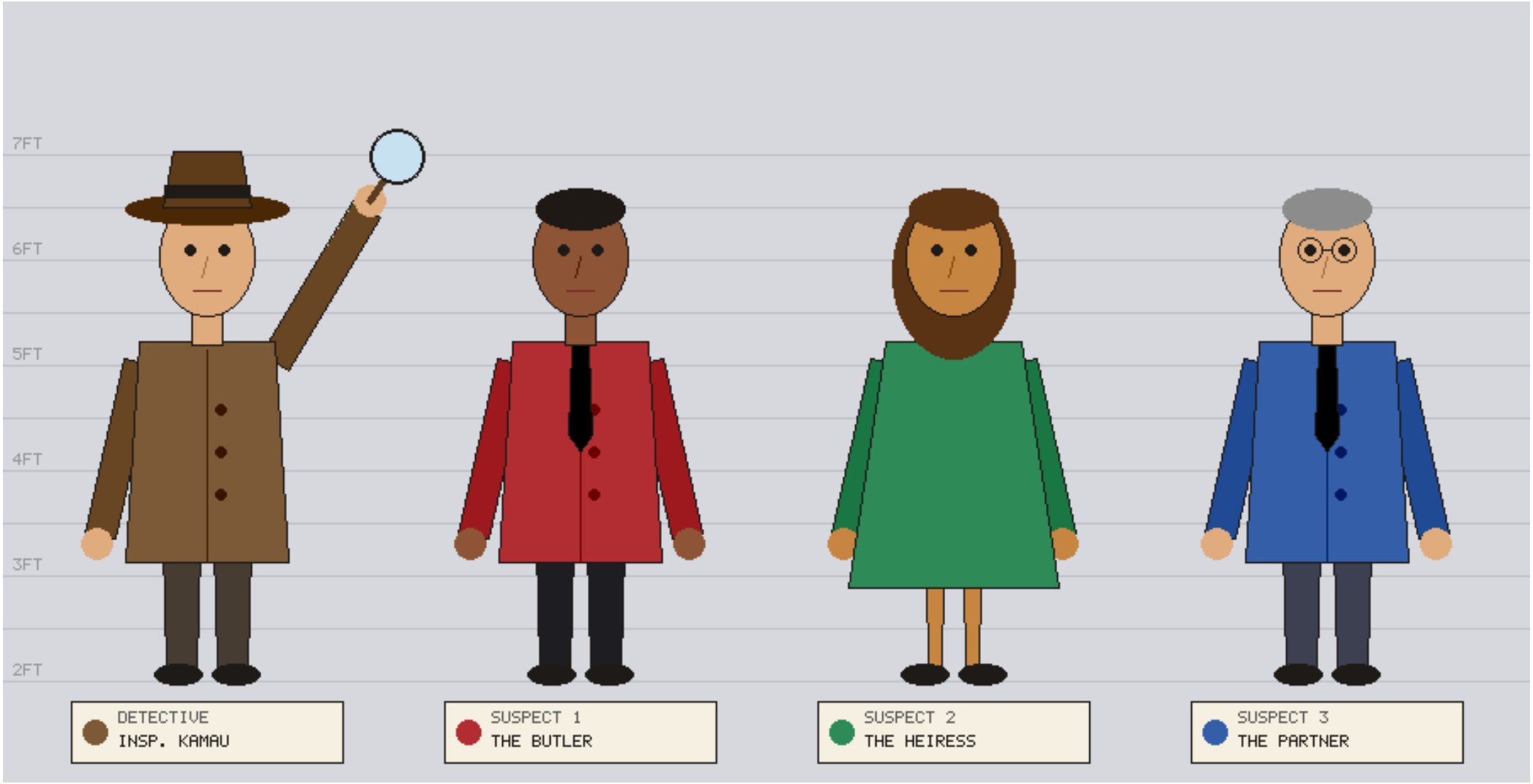
Implemented by Alvin | demos/10_crime_scene.cpp



The study where the crime happened, built entirely from our polygons, fills, circles, ellipses and lines. Yellow outlines mark interactive objects.

11. Project component: characters

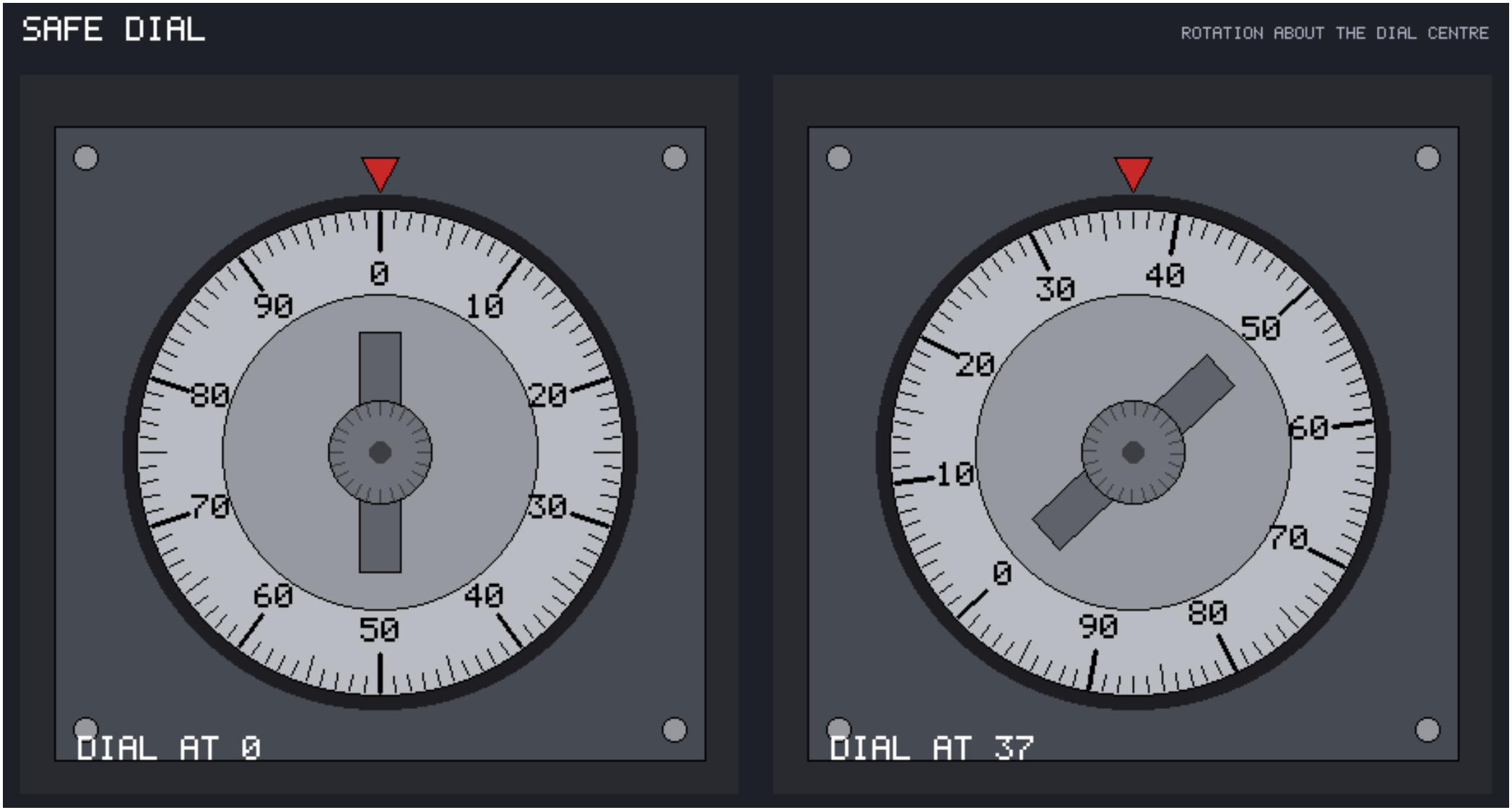
Implemented by Alvin | demos/11_characters.cpp



Detective plus three colour-coded suspects: ellipse heads, polygon coats, arms rotated about the shoulder, circle eyes and buttons.

12. Project component: safe dial puzzle

Implemented by Sudheysi | demos/12_safe_dial.cpp



100 ticks, numbers and the grip are placed by rotating points about the dial centre; the same code at two angles.

13. Project component: torn letter puzzle

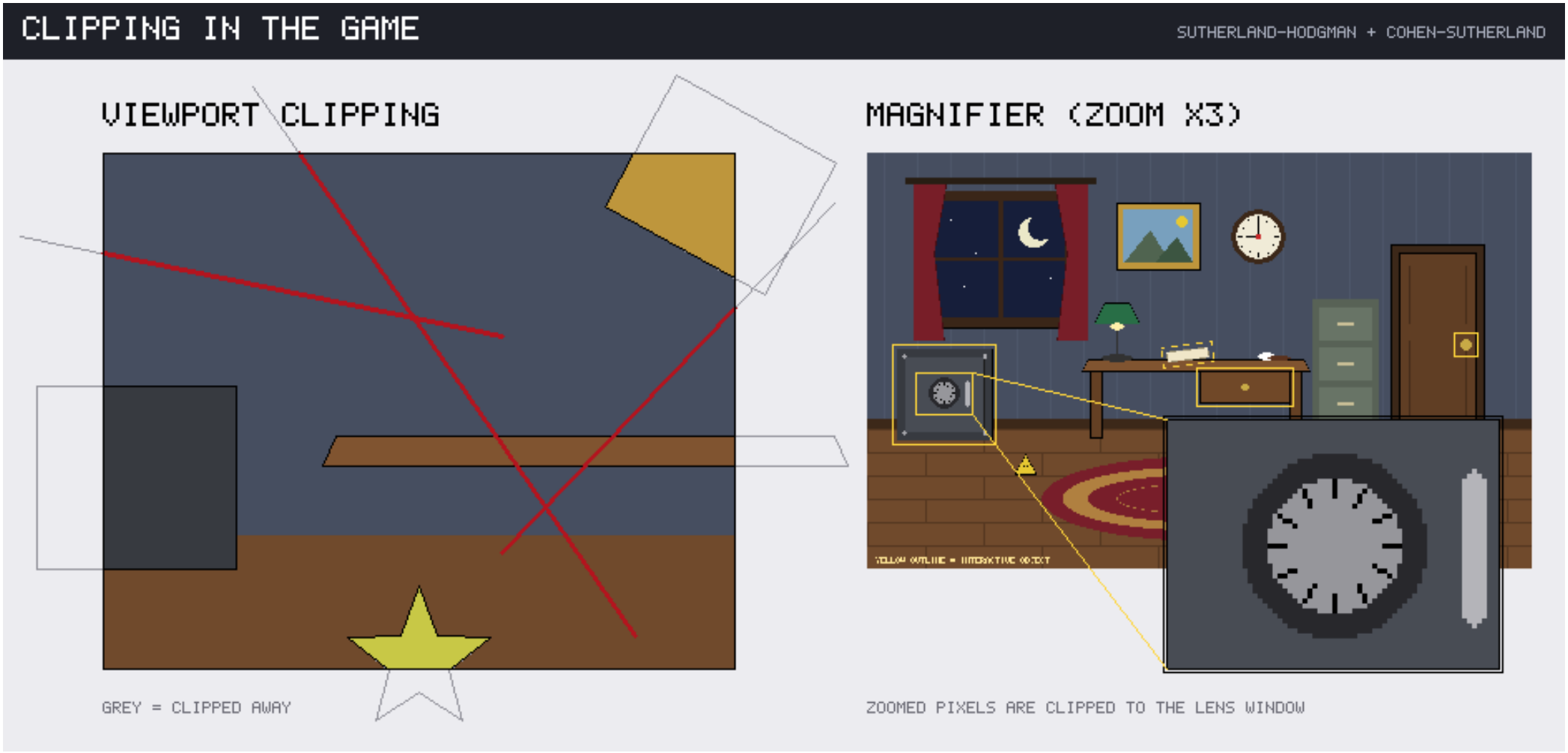
Implemented by Sudheysi | demos/13_torn_document.cpp



Two jagged tears cut the letter into 4 polygons. Each piece is rotated/translated, and pixels are mapped back with the inverse matrix.

14. Project component: clipping in the game

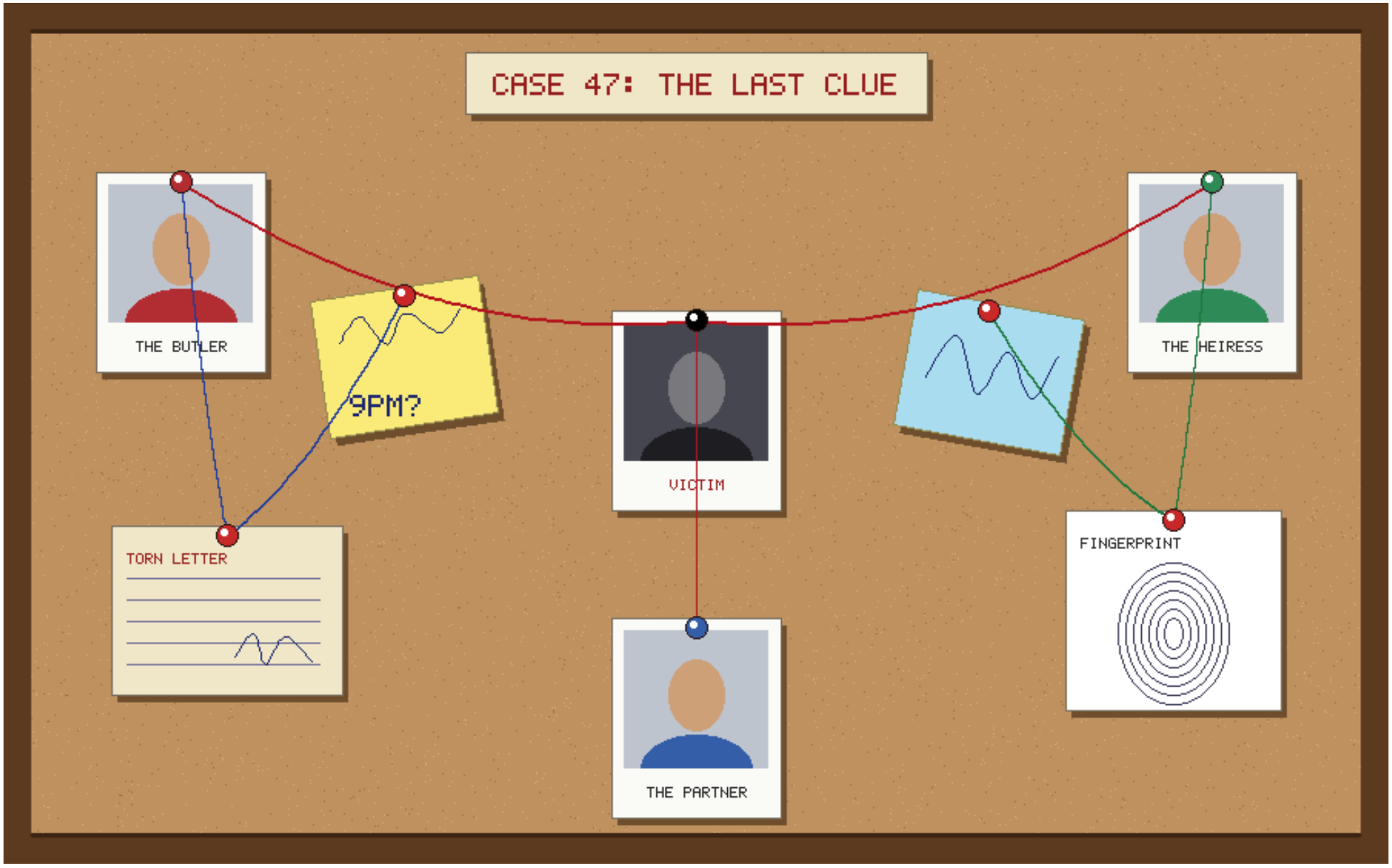
Implemented by Sudheysi | demos/14_clipping_scene.cpp



Furniture (Sutherland-Hodgman) and strings (Cohen-Sutherland) clipped to the viewport; the magnifier shows a 3x zoom clipped to its lens.

15. Project component: evidence board

Implemented by Latiffa | demos/15_evidence_board.cpp



Cork board with suspect photos, notes and pins; connections are sagging quadratic Bezier strings, and handwriting is Catmull-Rom.

16. Project component: game window layout

Implemented by Latiffa | demos/16_full_layout.cpp



The Case File panel (suspects and evidence slots) next to the Crime Scene viewport: the planned main screen.